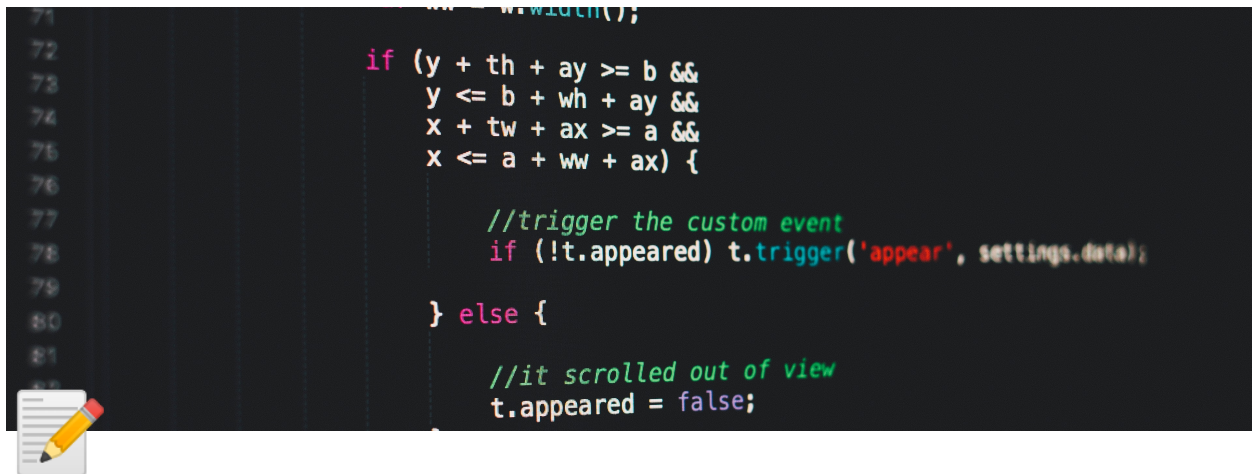




MEP



Falls ein Unit-Testfall nicht funktioniert, diesen auf @Disabled setzen

Zugriffsmodifizierer

`public` → überall sichtbar

`protected` → Zugriff im selben Package und zusätzlich alle Unterklassen (bzw. deren Konstruktor)

`Package` → alle im selben Package

`private` → nur in dieser Klasse



So verschlossen wie möglich und so offen wie nötig!

equals()

- Ist in Basisklasse Object enthalten
- Vergleicht aktuelles objekt mit anderem Objekt (je nach sinnhaftigkeit, hier Person mit ID)
- Benötigt hashCode implementation

```
@Override
public final boolean equals(Object obj) {
    if (obj == this) {
        return true;
    }
    if (!(obj instanceof Person)) {
        return false;
    }
    final Person other = (Person) obj;
```

```
    return this.id == other.id; //gibt bei gleicher ID true aus, sonst false
}
```

Test:

```
@Test
void testEqualsContract(){
    EqualsVerifier.forClass(oop_exam.Student.class).suppress(Warning.ALL_FIELDS_SHOULD_BE_USED).verify();
}
```

toString()

- Basisklasse von Object enthalten
- ermöglicht einfacheres debuggen durch ausgabe des Objekt in einem String
- möglichst immer Überschreiben

```
@Override
public String toString() {
    return "ID:" + id + "Vorname:" + vorname + "Nachname:" + nachname;
}
```

hashCode()

- von Basisklasse Object
- Nur bei 100% gleichem Objekt gleich → `Person person2 = new Person(person1)`

```
@Override
public final int hashCode() {
    return Objects.hash(this.id);
}
```

Test:

```
@Test
void testHashCodeTrue(){
    assertEquals(true, new Person(1,"Hans" ,"Huber").hashCode() == new Person(1,"Hans", "Huber").hashCode());
}
```

compareTo()

- implements Comparable<Person>
- compareTo() muss überschrieben werden

```
public final class Person implements Comparable<Person> {
    privat final long id;
    @Override
    public int compareTo(Person o) {
        return Long.compare(this.Id, o.Id);
    }
}
```

Test: Mindestens 3 Testfälle

```
@Test
void testCompareToBigger() {
    assertEquals(1, new Person(2, "Hans" , "Huber").compareTo(new Person(1, "Anna", "Meier")));
}
@Test
void testCompareToEqual() {
    assertEquals(0, new Person(1, "Hans" , "Huber").compareTo(new Person(1, "Hans" , "Huber")));
}
@Test
void testCompareToSmaller() {
    assertEquals(-1, new Person(1, "Hans" , "Huber").compareTo(new Person(2, "Anna", "Meier")));
}
```

Comperator<class>

- implements Comparator<Person>
- equals() und compare() muss überschrieben werden

```
public class PersonNameComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        int compare = p1.getNachname().compareTo(p2.getNachname());
        if (compare == 0) {
            compare = p1.getVorname().compareTo(p2.getVorname());
        }
        return compare;
    }
}
```

Collections - Datenstrukturen

List:

```
private List<Kurs> besuchteKurse = new ArrayList<>();
```

collection:

- Kann direkt auf die einzelnen Elemente Zugreifen (index beginnt bei 0)

```
//Generiere ArrayList mit 3 Werten
final Collection<Temperatur> verlauf = new ArrayList<>();
verlauf.add(new Temperatur(10.2f));
verlauf.add(new Temperatur(15.8f));
verlauf.add(new Temperatur(21.3f));
System.out.println("Anzahl Messwerte: " + verlauf.size());

//Ändere einer der eingefügten Werte
((List<Temperatur>) verlauf).set(1, new Temperatur(18.5f));
//bekomme den 2. Wert und printe ihn
final Temperatur t2 = ((List<Temperatur>) verlauf).get(1);
System.out.println("2. Messwert: " + t2.getCelsius());
```

- Wird nur add(); benötigt, so reicht der Typ `Collection`
- Alternativ kann auch `LinkedList<>()` verwendet werden

Set - Beispiel mit HashSet<>()

- Jedes Objekt kann nur ein einziges Mal enthalten sein (Gleichheit per Equals und hashCode)
- Objekte sollten nicht verändert werden
- Duplikat einfügen misslingt und liefert `false` zurück

```
final Set<Temperatur> verlauf = new HashSet<>();
verlauf.add(new Temperatur(10.2f));
verlauf.add(new Temperatur(15.8f));
verlauf.add(new Temperatur(21.3f));

//Einfügen von duplikat wird false liefern
final boolean result = verlauf.add(new Temperatur(15.8f));
```

Mögliche Implementationen: `HashSet`, `LinkedHashSet`, `TreeSet`, `EnumSet` usw.

Queue <>

- Schlange, nach FIFO - Prinzip (First in First out)

```
final Queue<Temperatur> queue = new ArrayBlockingQueue<>(5);
queue.offer(new Temperatur(10.2f));
queue.offer(new Temperatur(15.8f));
queue.offer(new Temperatur(21.3f));

//herausnehmen
System.out.println("1. Wert: " + queue.poll());
```

- Mögliche implementationen: `ArrayDeque`, `DelayQueue`, `LinkedBlockingQueue`, `PriorityQueue` usw.

Deque<>

- Ähnlich wie queue, kann jedoch in beide Richtungen verwendet werden

Map <K, V>

Spezielle Struktur für Schlüssel-Wert-Paare (Key-Value-pairs)

- Ein Schlüssel muss innerhalb einer Map eindeutig sein
- Wert als auch Schlüssel dürfen null

```
final Map<String, Temperatur> map = new HashMap<>();
map.put("first", new Temperatur(10.2f));
map.put("second", new Temperatur(15.8f));
map.put("third", new Temperatur(21.3f));
//get
System.out.println("2. Wert: " + map.get("second"));
```

- Mögliche Implementationen: `EnumMap`, `HashMap`, `Hashtable`, `Properties`, `TreeMap`

enum

- Benötigt eingene getter Methode

```
public enum Note{
    A(6.0f), B(5.50f), C(5.00f), D(4.50f), E(4.00f), F(3.00f);

    private float value;
    private Note(final float note){
        this.value = note;
    }
    public float getValue(){ //returnt bei input 5.5 "B"
        return this.value;
    }
}
```

Test:

```
@Test
void testEnum() {
    assertEquals(5.5f, Note.B.getValue());
    assertEquals(6f, Note.A.getValue());
}
```

Iterator<>

- Läuft nur Vorwärts

```
Iterator<Temperatur> iterator = list.iterator();
while(iterator.hasNext()) {
    final Temperatur t = iterator.next();
    t.doSomething();
}
```

- kein Einfügen neuer Elemente.

- keine Änderung enthaltender Elemente.
- kein Entfernen enthaltener Elemente.

`remove()`

Manipulation von Datenstrukturen (sortieren)

```
final List<Temperatur> list = new ArrayList<>();
// Sortiert Liste aufsteigend (natürliche Ordnung)
Collections.sort(list);
// Sortiert Liste absteigend (natürliche Ordnung)
Collections.sort(list, Collections.reverseOrder());

// Bestimmt das grösste Objekt (über Comparable).
final Temperatur maxima = Collections.max(list);
// Bestimmt das kleinste Objekt (über Comparable).
final Temperatur minima = Collections.min(list);
// Zählt wie oft ein Objekt (hier maxima) auftritt.
final int count = Collections.frequency(list, maxima);
```



`compareTo()` von `Comparable` interface muss implementiert sein

Exception Handling

- `throw` → Wirft eine exception
- `throws` → wirft eine exception an die näch höhere Ebene weiter

```
public Student (long idp, String vor, String nach){
    if(ID_MIN<this.id){
        throw new IllegalArgumentException("Es sind keine Werte unter 20200400000 für die ID erlaubt");
    }
    else if(nachname == null) {
        throw new NullPointerException("Nachname darf nicht leer sein");
    }
    else if(nach.length()<2){
        throw new IllegalArgumentException("Der Nachname muss mindestens zwei Zeichen enthalten");
    }
    //Kein If trifft beispielsweise zu
    this.id= idp;
    this.vorname = vor;
    this.nachname = nach;
}
```

Test:

```
@Test
public void throwsException1() {
    final Exception e = assertThrows(IllegalArgumentException.class, () -> {
        new Student(2020, "Peter", "Hans");
    });
}
```

```

    assertEquals("Zahl ist unter dem minimalwert", e.getMessage());
}

```

Try Catch

```

try{
    dosomething();
} catch (IOException e){
    return "";
}

```

Tripple A - Testing

- Arrange (Erstellen der Testobjekte)
- Act (Manipulieren der Testobjekte)
- Assert (Verifikation der Testergebnisse)

```

void testAufgabe5f() { //testing nach Tripple A
    Student hans = new Student(20200400000L, "Peter", "Hans");
    hans.addKurs("Kurs1", 3);
    hans.addKurs("Kurs2", 4);
    hans.addKurs("Kurs3", 5);
    final float avg = hans.avgNote();
    assertEquals(4, avg, 0.01);
}

```

⇒ jeden scheiss einzeln aufschreiben

Eventlistener

Eventquelle 1:

```

// Datenstruktur zur Speicherung aller Listener.
private final List<PropertyChangeListener> changeListeners = new ArrayList<>();
/**
 * Registriert einen PropertyChangeListener.
 * @param listener PropertyChangeListener.
 */
public void addPropertyChangeListener(final PropertyChangeListener listener) {
    this.changeListeners.add(listener);
}
/**
 * Deregistriert einen PropertyChangeListener.
 * @param listener PropertyChangeListener.
 */
public void removePropertyChangeListener(final PropertyChangeListener listener) {
    this.changeListeners.remove(listener);
}

```

Eventquelle 2:

```

public void switchOn() {
    if (isSwitchedOff()) {
        this.state = MotorState.ON;
        final PropertyChangeEvent pcEvent = new PropertyChangeEvent(this, "state", MotorState.OFF, MotorState.ON);
        this.firePropertyChangeEvent(pcEvent);
    }
}
/**
 * Informiert alle PropertyChangeListener über PropertyChangeEvent.
 * @param pcEvent PropertyChangeEvent.
 */
private void firePropertyChangeEvent(final PropertyChangeEvent pcEvent) {
    for (final PropertyChangeListener listener : this.changeListeners) {
        listener.propertyChange(pcEvent);
    }
}

```

Empfänger:

```

public final class Car implements PropertyChangeListener {
    public Car(final String model) {
        // Komponenten erzeugen.
        this.motor = new Motor();
        this.lightFrontLeft = new Light();
        this.lightFrontRight = new Light();
        // Sich selber als Listener registrieren.
        this.motor.addPropertyChangeListener(this);
        this.lightFrontLeft.addPropertyChangeListener(this);
        this.lightFrontRight.addPropertyChangeListener(this);
    }
    ...
}

```

Event-Behandlung:

```

public final class Car implements PropertyChangeListener {
    ...
    @Override
    public void propertyChange(final PropertyChangeEvent event) {
        if (event.getSource() == this.motor) {
            this.handleMotorEvent("Motor", event);
        }
        if (event.getSource() == this.lightFrontLeft) {
            this.handleLightEvent("Scheinwerfer Links", event);
        }
        if (event.getSource() == this.lightFrontRight) {
            this.handleLightEvent("Scheinwerfer Rechts", event);
        }
    }
}

```

Namensconvention:

- `XxxEvent` – Klasse für den Event
- `XxxListener(...)` – Interface für den Listener

- `xxx[Performed](...)` – Methode auf Listener-Interface
- `addXxxListener(...)` – Registrierung von Listnern auf Quelle
- `removeXxxListener(...)` – Deregistrierung von Listnern
- `fireXxxEvent(...)` – Versenden von Events von Quelle

Scanner

```
import java.util.Scanner;  
...  
Scanner scanner = new Scanner(System.in);  
System.out.println("Bitte Temperatur eingeben ('exit' zum Beenden): "); //durch LOG ersetzen  
input = scanner.next();
```